

PulseLM Biosignal Analysis Pipeline

Carolina Dias*, Elif Öztürk*, Leonor Couto*, Mariana Pereira*, Simão Bernardo*, Sofia Fernandes*

* Faculty of Engineering, University of Porto (FEUP)

Emails: up202500534@up.pt, up201609047@up.pt, up202205796@up.pt,
up202205093@up.pt, up202502529@up.pt, up202502530@up.pt

Abstract—The importance of data continues to grow as increasingly large volumes are generated and stored every day. Within this scope, efficiently processing and analyzing big data is equally relevant and challenging, requiring distributed computing frameworks and scalable machine learning pipelines. Datasets such as PulseLM provide a suitable environment for evaluating the performance, scalability, and computational efficiency of modern big data systems. This report presents a distributed machine learning pipeline built on the PulseLM dataset using Apache Spark and Google Cloud Dataproc. Raw PPG signals are transformed into compact statistical feature representations and used across multiple binary clinical classification tasks. Several Spark MLlib models and a PyTorch multi-task neural network are evaluated. The best-performing models achieve macro-F1 scores up to 0.931, while distributed exploratory data analysis reaches a 2.23× speed-up when scaling from two to four workers, though overall pipeline speedup remains modest at 1.12×, dominated by training time. Overall, the results highlight the trade-offs between scalability, computational efficiency, and predictive performance in large-scale machine learning workflows.

I. INTRODUCTION

The digital world has transformed the way data is collected and stored. As the value of data continues to grow, increasingly large volumes of information are generated and stored every minute [1], [2]. To keep up with this digital transformation, big data engineering has become a highly relevant field in recent years. However, handling big data also introduces significant technical challenges, including scalability in storage, distributed processing, fault tolerance, and computational efficiency [3]. To address these challenges, distributed computing frameworks such as Apache Spark and cloud-based platforms such as Google Cloud Dataproc have been developed. These technologies enable parallel processing of large-scale datasets while reducing execution time, making them essential tools in modern data engineering pipelines [4], [5], [6].

An important topic in big data engineering is resource optimization. Current research increasingly focuses not only on predictive performance, but also on computational efficiency and scalability when models are deployed on large datasets [1], [3]. Thus, evaluating the performance of distributed pipelines has become a key aspect of big data projects, particularly in environments where scalability and efficiency are critical.

Within this scope, the PulseLM dataset provides a relevant case study for big data analytics and modelling. The dataset contains physiological measurements and is characterized by a high-dimensional structure, typically comprising tens of features and over a million samples. Its complexity and structure make it suitable for investigating how big data engineering

solutions can support efficient modelling while addressing efficiency and scalability challenges [7].

The goal of this project is therefore to design and implement a big data pipeline using the PulseLM dataset ¹, leveraging distributed technologies such as Apache Spark and Google Cloud Dataproc. The project focuses on building an efficient and scalable workflow for real-world machine learning (ML) tasks, including data ingestion, preprocessing, feature transformation, model training, and evaluation. The main end-goal is to analyze the scalability and computational efficiency of the pipeline in a distributed environment, while also assessing its practical benefits and limitations in large-scale data processing scenarios.

II. RELATED WORK

Recent advances in physiological foundation models have accelerated multimodal learning for photoplethysmography (PPG). PulseLM [7] introduced the first large-scale PPG-text benchmark for physiological question answering, containing over one million standardized PPG segments, twelve downstream tasks, and approximately 2.5 million QA pairs. It also established standardized preprocessing and evaluation protocols for multimodal PPG-aware LLMs.

Our work builds directly on PulseLM by leveraging its harmonized QA annotations for classical ML analysis using nine handcrafted signal features. Among its source datasets, VitalDB [8] contributes approximately 163,000 high-fidelity PPG segments with synchronized ECG references, providing clinically reliable ground truth.

The comparison between classical feature engineering and deep learning is motivated by foundational work in physiological signal analysis. Hannun et al. [9] demonstrated cardiologist-level arrhythmia detection using convolutional neural networks, while recent PPG foundation models such as PaPaGei [10] and PulsePPG [11] achieved strong performance across diverse PPG tasks, emphasizing the importance of model architecture and dataset diversity.

Despite these advances, classical signal-processing approaches remain valuable because of their interpretability and efficiency. Kachuee et al. [12] demonstrated cuffless blood pressure estimation using handcrafted morphological and temporal PPG features, many of which remain widely used in resource-constrained settings.

¹Data can be found in here.

Several datasets incorporated into PulseLM also reveal important limitations in current physiological benchmarks. The pediatric sleep-disordered breathing dataset by Garde et al. [13] exhibits significant class imbalance, while WESAD [14], the primary stress-detection dataset within PulseLM, contains relatively few labeled examples. These constraints suggest that performance limitations often arise not only from model capacity but also from dataset composition and label scarcity.

The field is increasingly shifting toward language-based physiological reasoning. Q-HEART [15] reformulated ECG interpretation as a multimodal question-answering task, while PulseLM extended this paradigm to PPG across twelve physiological tasks. More broadly, SensorLM [16] proposed unified signal-text representations across wearable sensing modalities, situating PPG understanding within the broader context of multimodal physiological foundation models.

Within this landscape, our work provides a classical ML baseline on PulseLM, bridging interpretable physiological signal processing with emerging multimodal foundation models. By evaluating handcrafted PPG features under a standardized benchmark, we identify where classical methods remain competitive and where dataset limitations constrain all approaches equally.

III. DATASET PROFILING

A. Dataset Overview and Ingestion

PulseLM is hosted on HuggingFace (Manhph2211/PulseLM) and comprises 16 named configurations (sub-datasets). One configuration (*afppgecg*) was skipped during ingestion due to degenerate label structure: its *qa* field exclusively produces *non_af* labels, contributing no positive AF examples and yielding null labels for all other tasks, making it unsuitable for any of the ten binary classification tasks. The remaining 15 were processed across train, validation, and test splits. The ingested corpus totals 939,126 rows: 728,621 train, 82,337 validation, and 128,168 test. Each row represents one 10-second PPG segment, stored as a Parquet file on disk with a schema of 25 columns: 9 engineered signal features (i.e., *f_mean*, *f_std*, *f_min*, *f_max*, *f_p2p*, *f_skew*, *f_kurt*, *f_zcr*, *f_dom_freq*), three meta-data fields (i.e., *signal_len*, *text_len*, *qa_count*), the dataset identifier, and 12 nullable label columns (i.e., *lbl_HR*, *lbl_BP*, *lbl_SQI*, *lbl_Stress*, *lbl_SDB*, *lbl_HRV-SDNN*, *lbl_HRV-RMSSD*, *lbl_HRV-pNN50*, *lbl_AF*, *lbl_Arrhythmia*, *lbl_SpO2*, *lbl_RR*). The memory footprint is modest: approximately 373.8 MB for train, 42.2 MB for validation, and 65.7 MB for test.

B. Sub-dataset Composition and Signal Characteristics

The corpus is highly imbalanced across contributing sub-datasets. Table I summarises the training-split row counts. The three largest sources, *sdb* (205,618), *wildppg* (180,000), and *vitaldb* (131,324), account for roughly 71% of all training rows, while the smallest (*bcg*: 521, *ppgbp*: 294) are nearly negligible by volume. This extreme size disparity has direct consequences for task-specific label availability:

clinical tasks are typically covered by only one or two sub-datasets (e.g., Stress exclusively by *wesad*, AF exclusively by *mimicperform* and *ppgarrhythmia*), making their training pools dependent on a single source’s size.

TABLE I
TRAINING ROWS PER SUB-DATASET (TOP 8 SHOWN).

Sub-dataset	Train rows
<i>sdb</i>	205,618
<i>wildppg</i>	180,000
<i>vitaldb</i>	131,324
<i>uci</i>	89,054
<i>ppgarrhythmia</i>	36,820
<i>dalia</i>	29,294
<i>uqvitalsigns</i>	25,202
<i>utsappg</i>	12,740

All sub-datasets are standardised to a fixed signal length of 1,250 samples per segment. Average QA pair counts per sample range considerably: *vitaldb* averages 546 QA pairs per row versus *mimicperform* at 94, reflecting variation in annotation depth across sources.

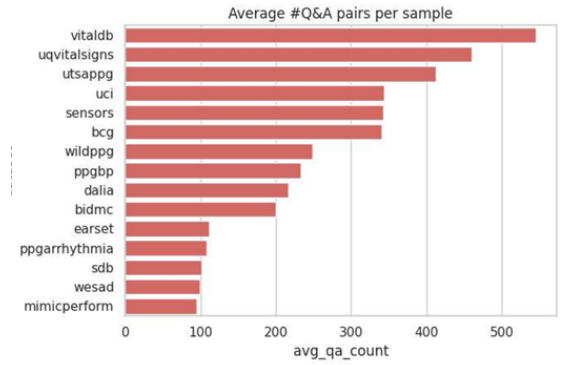


Fig. 1. Rows per sub-dataset and average QA count per sample

C. Schema Validation and Data Quality

A schema validation step confirmed all nine expected feature columns are present in the Parquet schema. The null/NaN filter removed zero rows from the training split maintaining 728,621 rows, indicating that the feature extraction stage produced no missing or infinite values for any retained segment. A Signal Quality Index (SQI) gate (threshold 0.5) was defined in the pipeline to discard low-quality segments, but the *f_sqi_score* column was absent from this Parquet version, so the gate was skipped. Notably, *lbl_SQI* returned zero labelled rows in all splits, confirming that SQI labels were not populated during ingestion. No duplicate detection was explicitly performed, but since each row maps to a distinct streamed sample from HuggingFace with a per-batch write, duplicates are not expected within splits.

D. Label Coverage and Class Imbalance

Since PulseLM aggregates heterogeneous sources, label coverage is sparse and task-specific. Table II reports the

fraction of training rows carrying a non-null label per task and sub-dataset for the most informative configurations.

TABLE II
LABEL COVERAGE (FRACTION OF TRAIN ROWS PER CONFIG, SELECTED TASKS).

Config	HR	BP	SDB	HRV	AF
vitaldb	100%	100%	0%	100%	0%
sdb	0%	0%	100%	0%	0%
ppgarrhythmia	0%	0%	0%	0%	100%
wesad	0%	0%	0%	0%	0%
uqvitalsigns	100%	99.2%	0%	100%	0%

This one-to-one mapping between tasks and source datasets is a defining structural property of PulseLM. Label counts in the training split span four orders of magnitude: Heart Rate (HR) has 470,920 labelled rows while Stress has only 2,104 and Atrial Fibrillation (AF) has 3,239. The binary class distributions within labelled rows are also imbalanced. For HR Variability (HRV) tasks, the "low" class dominates: 78.9% of HRV-SDNN labels are "low", 61.2% of HRV-RMSSD, and 68.8% of HRV-pNN50. Stress is comparatively balanced (baseline: 44.3%, stress: 24.4%, meditation: 19.7%, amusement: 11.6%). SDB is moderately imbalanced with 63.8% of samples classified as normal (AHI < 5). SpO₂ is degenerate, with 100% of its 15,728 labelled rows classified as "normal", making it unsuitable for binary classification. These imbalances directly motivated the inverse-frequency class weighting strategy described in Section IV-A.

E. Feature Characterisation and Discriminability

The nine signal features are all double-precision floats. A distributed F-ratio analysis, computing between-sub-dataset variance of feature means divided by pooled within-sub-dataset standard deviation via a single `groupBy("dataset").agg(avg, stddev)` Spark pass, ranks features by their ability to separate sub-datasets without fitting any model. The dominant frequency (`f_dom_freq`) scores highest with $F \approx 6.07$, followed by kurtosis ($F \approx 1.93$), minimum ($F \approx 0.79$), skewness ($F \approx 0.38$), and mean ($F \approx 0.37$). The remaining features (`std`, `max`, `p2p`, `zcr`) have lower discriminative F-ratios. The strong separation by `f_dom_freq` is physiologically intuitive: different acquisition setups (wrist, finger, clinical) and subject populations produce PPG waveforms at distinct dominant frequencies. Box-plots of the top-5 features across sub-datasets confirm heterogeneous distributions, particularly for `f_dom_freq` and `kurt`, motivating the per-feature `StandardScaler` normalisation applied in the ML pipeline, as seen in Figure 2.

F. Key Insights

Several structural observations from profiling directly shaped the modelling choices described in Section IV-A. First, label scarcity, not feature quality or model capacity, is the primary limiting factor for tasks such as AF (3,239 train rows), Stress (2,104), and Respiratory Rate (RR) (31,570). Second,

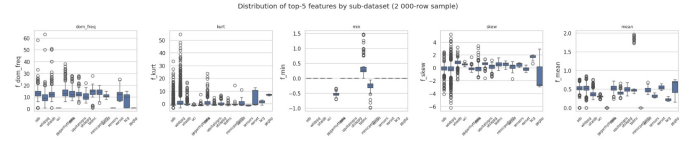


Fig. 2. Box-plots of top-5 discriminative features by sub-dataset

the per-task, single-source data provenance means that model generalisation is bounded by each source's domain: a Stress classifier trained solely on WESAD wristband data cannot generalise to clinical PPG contexts. Third, the absence of SQI scores in the current Parquet version means that a non-trivial fraction of potentially noisy segments may remain in all splits, representing a data quality risk for future work. Finally, the uniformity of signal length (all segments are exactly 1,250 samples) eliminates padding artefacts as a confound and simplifies the VectorAssembler ingestion pipeline.

IV. MACHINE LEARNING

A. Task description

We framed the learning problem as ten binary classification tasks taken from PulseLM's clinical QA vocabulary (originally PulseLM includes twelve). We dropped SQI, which was already enforced as a preprocessing gate (segments with $SQI < 0.5$ never reach the model), and SpO₂, which was too thinly populated across the contributing sub-datasets to give us a usable split. What's left (HR, BP, HRV-SDNN, HRV-RMSSD, HRV-pNN50, AF, Arrhythmia, SDB, Stress, RR) spans the cardiac, vascular, autonomic, and respiratory channels that PPG actually carries information about.

Every task collapses PulseLM's multi-class answer set into a normal-vs-abnormal label using a fixed mapping. HR sends "normal" to 0 and both "bradycardia" and "tachycardia" to 1. SDB sends anything starting with "normal" to 0 and the three apnea-severity strata to 1. AF treats "af" as positive and "non_af" as negative. We created binary labelling for two reasons. First, several of the multi-class answer sets in PulseLM are badly long-tailed: BP, for example, is mostly "normal", with a handful of rows in the hypertensive-crisis stratum: per-class training would have essentially nothing to fit on for those rare strata. Second, the clinical screening question that PPG is meant to answer is itself binary: does this segment warrant follow-up?

On the query side, three distributed Spark aggregations carry the weight. A per-task class-balance query reports row counts and positive-class fraction across the sixteen sub-datasets; the inverse-frequency weights used by RF-whole come straight out of it. A label-coverage matrix (one row per config, one column per task, each cell the fraction of rows carrying that label) makes the provenance of each task explicit: WESAD feeds Stress, VitalDB feeds HR and BP, the AF-PPG-ECG corpus feeds AF. An F-ratio pass computes $F_j \approx \text{Var}_{\text{between}}(\mu_j)/\bar{\sigma}_j$ for each of the nine features in a single `groupBy("dataset").agg(avg,`

stddev) call, ranking features by how well they separate sub-datasets without fitting anything.

Each ten-second PPG segment is reduced to a nine-dimensional vector $\{\text{mean}, \sigma, \text{min}, \text{max}, \text{p2p}, \text{skew}, \text{kurt}, \text{zcr}, f_{\text{dom}}\}$, with f_{dom} the dominant FFT bin of the mean-centred signal. The descriptors are intentionally simple and interpretable: peak-to-peak amplitude and dominant frequency map onto pulse morphology and beat rate in ways a clinician can reason about, and the remaining statistics can be computed in a single streaming pass during ingestion. The objective is not to surpass PPG foundation models in raw F1, but to evaluate the performance limits of classical feature representations on the same benchmark. [12].

Four classifiers train on this same nine-feature input. RF-subset fits one Spark `RandomForestClassifier` per task on just the rows where the task label is present. RF-whole refits the same RF on the full training `DataFrame`, with a per-task weight column w_t that is zero on unlabelled rows and $w_c = N/(2N_c)$ on labelled rows; this lets us bolt inverse-frequency class weights onto the same pipeline without re-partitioning the data once per task. Spark MLP is a `MultilayerPerceptronClassifier` with layers $[9, 16, 8, 2]$, one per task, and exists to ask whether a small feed-forward net catches anything non-linear that the RF misses. The PyTorch multi-task NN has a shared trunk $9 \rightarrow 64 \rightarrow 32$ feeding ten sigmoid heads, trained on the pandas matrix with masked binary cross-entropy. The PyTorch multi-task network was implemented outside Spark because Spark MLlib does not support multi-output classification natively. Every classifier in the spark.ml API is designed for a single label column, with no mechanism to share representations across tasks or train multiple heads jointly. The canonical Spark workaround is to loop over tasks and fit one independent model per label column. PyTorch was therefore the tool chosen for the multi-task setting, allowing full control over the shared trunk architecture, at the cost of operating outside the distributed Spark execution engine. Since all four read the same feature vectors, any gap in macro-F1 between them is informative on its own: it tells us whether the win sits in how we composed the training set, in the model family, or in coupling the tasks together. Algorithm 1 shows the abridged Spark training routine that both RF variants use; the MLP variant just swaps the estimator.

To know whether any of the models are doing real work, we also compute the analytical majority-class macro-F1 per task. A constant predictor that always emits the majority class with frequency π has precision π , recall 1, and macro-F1 $\pi/(\pi + 1)$ averaged across the two classes. Reporting lift above this floor matters because the weighted F1 reported by `MulticlassClassificationEvaluator` collapses to plain accuracy under constant predictions; comparing it against the models' macro-F1 would compare two different metrics. That is why the next subsection uses macro-F1 throughout.

Algorithm 1 Per-task Spark Training

```

1:  $ycol \leftarrow "y_{\{\text{task}\}}"$ 
2: if  $use\_weights$  then
3:   Replace null values in  $ycol$  with 0 in  $df\_train$ 
4:    $df\_tr \leftarrow df\_train$ 
5:    $weight\_col \leftarrow "wt_{\{\text{task}\}}"$ 
6: else
7:   Filter rows where  $ycol$  is not null
8:    $df\_tr \leftarrow df\_train$ 
9:    $weight\_col \leftarrow \text{None}$ 
10: end if
11: Define pipeline with stages:
    • VectorAssembler( $feature\_cols \rightarrow raw\_features$ )
    • StandardScaler( $raw\_features \rightarrow features$ )
    • RandomForestClassifier( $features, ycol, weight\_col,$ 
       $numTrees = 50, maxDepth = 15)$ 
12:  $model \leftarrow pipeline.fit(df\_tr)$ 

```

B. Results

Table III reports test macro-F1 across the four models. Tasks with large labelled pools achieved strong results: HR, HRV-SDNN, and Arrhythmia reached macro-F1 scores of 0.859, 0.899, and 0.931 under RF-subset, respectively. Data-scarce tasks performed worse: AF yielded a macro-F1 of only 0.328, and Stress reached 0.613 despite a relatively balanced class distribution, reflecting its small training pool of 2,104 rows.

TABLE III
PER-TASK TEST MACRO-F1 ACROSS FOUR MODELS.

Task	RF-sub	RF-whole	MLP	PyTorch NN
AF	0.328	0.590	0.436	0.572
Arrhythmia	0.931	0.911	0.917	0.805
BP	0.604	0.587	0.589	0.562
HR	0.859	0.820	0.842	0.681
HRV-RMSSD	0.817	0.369	0.809	0.389
HRV-SDNN	0.899	0.419	0.899	0.339
HRV-pNN50	0.839	0.382	0.839	0.355
RR	0.508	0.369	0.524	0.465
SDB	0.574	0.582	0.565	0.519
Stress	0.613	0.224	0.557	0.569

The RF-whole model produced notably degraded macro-F1 on the three HRV tasks relative to RF-subset, dropping from 0.899 to 0.419 for HRV-SDNN, from 0.817 to 0.369 for HRV-RMSSD, and from 0.839 to 0.382 for HRV-pNN50. The Stress task suffered the most severe degradation, falling from 0.613 to 0.224. These results show that diluting task-specific signal with a large volume of rows where the label is absent can harm minority-class recall, even when inverse-frequency reweighting is used. In contrast, RF-whole improved AF from 0.328 to 0.590, suggesting that broader data exposure benefits tasks where the task-specific subset is too small to generalise.

The Spark MLP performed similarly to RF-subset on well-represented tasks, achieving macro-F1 of 0.899 on HRV-SDNN, 0.917 on Arrhythmia, and 0.842 on HR. However, on low-resource tasks neither model offered a consistent advantage, with AF at 0.436 and Stress at 0.557. The PyTorch multi-task neural network consistently underperformed

both per-task Spark models across nearly all tasks. Its best results were on Arrhythmia (0.805) and HR (0.681), while HRV tasks degraded substantially, with HRV-SDNN falling to 0.339. This suggests that, at this feature dimensionality, shared representations across tasks do not provide sufficient benefit to outweigh the optimization difficulties introduced by joint training on heterogeneous tasks.

Table IV compares the best per-task Spark model against a majority-class baseline. All models produce meaningful lift on structurally easier tasks: HR achieves a lift of 0.430, Arrhythmia 0.530, and HRV-SDNN 0.419. In contrast, important but data-scarce tasks show limited lift: AF yields only 0.233 above baseline and Stress 0.255. This indicates that the main limitation in these cases is not model capacity, but the composition of the dataset itself. RR and SDB fall in an intermediate range, with lifts of 0.170 and 0.193 respectively, consistent with their moderate labelled pool sizes and the structural difficulty of distinguishing adjacent ordinal classes.

TABLE IV
MAJORITY-CLASS BASELINE VS. BEST SPARK MODEL (TEST MACRO-F1).

Task	Baseline F1	Best F1	Lift
AF	0.357	0.590	0.233
Arrhythmia	0.401	0.931	0.530
BP	0.366	0.604	0.238
HR	0.429	0.859	0.430
HRV-RMSSD	0.451	0.817	0.367
HRV-SDNN	0.480	0.899	0.419
HRV-pNN50	0.462	0.839	0.378
RR	0.354	0.524	0.170
SDB	0.389	0.582	0.193
Stress	0.358	0.613	0.255

Overall, macro-F1 performance correlates strongly with labelled sample size rather than model sophistication. Tasks with tens of thousands of labelled training rows consistently achieve macro-F1 above 0.80, while tasks with fewer than 5,000 labelled examples remain below 0.62 across all models. This pattern suggests that, within the handcrafted feature setting, increasing architectural complexity is unlikely to produce significant improvements unless the issue of label scarcity in underrepresented tasks is first addressed.

V. RUNTIME AND SCALABILITY ANALYSIS

To characterize the distributed execution performance of the pipeline, we measured end-to-end wall-clock time on Google Cloud Dataproc at two cluster sizes and decomposed it into three distinct phases: exploratory data analysis (EDA), preprocessing, and model training. Because the full PulseLM dataset proved too large for the Dataproc kernel to ingest without crashing, all benchmarks were conducted on a 1,000-row sample drawn uniformly at random from each configuration subset. While this curtails the absolute execution times relative to the full corpus, the sample is large enough to exercise the complete Spark execution graph - task scheduling, shuffle, and YARN resource negotiation - and the scaling ratios between cluster sizes remain valid indicators of distributed performance.

A. Methodology

Experiments were run on Google Cloud Dataproc under Spark 3.5.3 / YARN, with autoscaling disabled to ensure fully controlled and reproducible comparisons. Both the master and worker nodes used n2-highmem-2 machines (2 vCPUs, 16 GB RAM each) with pd-standard persistent disks (100 GB on the master, 200 GB per worker). Parquet data was stored on a GCS staging bucket and read by workers over the cluster’s internal default VPC (internal IP only). Table V summarises the fixed hardware configuration.

TABLE V
DATAPROC CLUSTER HARDWARE CONFIGURATION.

Parameter	Value
Image version	2.3.29-debian12
Region / Zone	us-west3 / us-west3-b
Master topology	1 master + N workers
Machine type	n2-highmem-2 (2 vCPU, 16 GB)
Master disk	100 GB pd-standard
Worker disk	200 GB pd-standard
GPUs	None
Autoscaling	Disabled
Secure boot / vTPM	Enabled

Two worker counts were evaluated: a two-worker cluster and a four-worker cluster. Both clusters were run with default Spark settings; the effective task-level parallelism increased between configurations through the addition of executor instances.

Three phases were timed independently using Python `time.perf_counter` walls around the triggering Spark *action* in each case:

- **EDA** - a `groupBy/agg` computing per-class feature means across all 9 signal features; representative of read-heavy analytical aggregations.
- **Preprocessing** - parquet ingestion from GCS, SQI/null filtering, and a `repartition` step that forces a full shuffle.
- **Training** - sequential fitting of all 10 per-task `RandomForestClassifier` pipelines (whole-dataset weighted variant) via `spark.ml`.

Each configuration was run once per cluster; the results were serialised to JSON and aggregated for comparison.

B. Results

Table VI reports the measured wall-clock times for each phase and cluster. Moving from two to four workers reduced total pipeline time from 55.2 s to 49.4 s, a wall-clock reduction of 10.6% ($\approx 1.12\times$ overall speedup).

The speedup is not uniform across phases. The EDA aggregation scaled the most favourably: doubling the executor count yielded a $2.23\times$ speedup (0.433 s $\rightarrow$ 0.194 s), approaching the theoretical linear ideal. This is consistent with Spark’s efficient partitioned `groupBy` execution, where additional executors reduce the per-partition work and the subsequent shuffle merge.

TABLE VI
PER-PHASE WALL-CLOCK TIMES (SECONDS) ON DATAPROC.

Phase	2-worker	4-worker
EDA	0.433	0.194
Preprocessing	6.180	5.886
Training	48.613	43.308
Total	55.226	49.388

Preprocessing showed a modest improvement ($1.05\times$), reflecting that the bottleneck here is largely I/O latency from GCS rather than compute - an overhead that additional cores cannot eliminate. Training, which dominates total time at $\approx 88\%$ of the wall clock, improved by $1.12\times$: the ten independent `RandomForestClassifier` fits each parallelise internally over Spark partitions, so the gain is real but sub-linear because tree-construction has sequential components that Amdahl's law constrains.

The aggregate speedup of $1.12\times$ for a $2\times$ increase in executor count reflects a *parallel efficiency* of ~ 0.56 - consistent with expectations for a mixed workload dominated by a training phase with limited inter-task parallelism. The results suggest that, at full dataset scale, further scaling to 8 or more workers would yield progressively diminishing returns on the training phase while EDA and shuffle-heavy preprocessing would benefit more substantially. A larger cluster would also mitigate the memory pressure that forced us to subsample the dataset, potentially enabling the pipeline to run over the complete corpus and making the absolute execution times more representative.

VI. CONCLUSION

This work developed a scalable ML pipeline for PulseLM-based physiological signal analysis using Apache Spark and Google Cloud Dataproc. By representing each 10-second PPG segment with nine handcrafted statistical and frequency-domain features, the pipeline enabled efficient classification across multiple clinical tasks while remaining computationally lightweight and interpretable.

The results show that classical feature-based models remain strong baselines for structured biomedical signals. The best per-task Random Forest models achieved high macro-F1 scores for Arrhythmia, HRV(-SDNN, -pNN50, -RMSSD), and HR, with Arrhythmia reaching 0.93 and HRV-SDNN reaching 0.89. These gains over majority-class baselines indicate that the models captured meaningful physiological patterns rather than relying only on class imbalance. At the same time, performance varied considerably across tasks. Blood pressure, sleep-disordered breathing, respiratory rate, stress, and atrial fibrillation achieved more modest macro-F1 scores.

From a systems perspective, the pipeline demonstrates the practicality of distributed big data tools for biomedical signal workflows. Apache Spark and Google Cloud Dataproc supported distributed ingestion, aggregation, preprocessing, and model training without requiring GPU infrastructure.

These findings highlight an important trade-off in large-scale physiological ML. Handcrafted statistical PPG features

combined with distributed classical models provide fast, interpretable, and resource-efficient baselines. However, their performance is strongly bounded by label availability, task provenance, and signal quality.

Thus, future work should focus on both data and model improvements. First, replacing the handcrafted feature set with one-dimensional convolutional or transformer-based encoders operating directly on raw PPG signals may improve representation learning, especially if combined with distributed training frameworks. Second, multi-task learning should be revisited with richer raw-signal embeddings and better task-balancing strategies. Finally, extending the batch-oriented Spark workflow to Spark Structured Streaming would allow the same architecture to support real-time physiological analytics.

REFERENCES

- [1] S. Sapiroglu and D. Sinanc, "Big data: A review," in *2013 international conference on collaboration technologies and systems (CTS)*. IEEE, 2013, pp. 42–47.
- [2] I. Yaqoob, I. A. T. Hashem, A. Gani, S. Mokhtar, E. Ahmed, N. B. Anuar, and A. V. Vasilakos, "Big data: From beginning to future," *International Journal of Information Management*, vol. 36, no. 6, pp. 1231–1247, 2016.
- [3] J. Fan, F. Han, and H. Liu, "Challenges of big data analysis," *National science review*, vol. 1, no. 2, pp. 293–314, 2014.
- [4] [Online]. Available: <https://cloud.google.com/storage>
- [5] B. Chambers and M. Zaharia, *Spark: The definitive guide: Big data processing made simple*. "O'Reilly Media, Inc.", 2018.
- [6] C. S. K. Sahith, S. Muppidi, and S. Merugula, "Apache spark big data analysis, performance tuning, and spark application optimization," in *2023 International Conference on Evolutionary Algorithms and Soft Computing Techniques (EASCT)*. IEEE, 2023, pp. 1–8.
- [7] H. M. Pham, J. Wu, X. Ma, Y. Zhang, Y. Xu, A. Saeed, B. Zhu, Z. Pan, and D. Ma, "PulseLM: A foundation dataset and benchmark for ppg-text learning," *arXiv preprint arXiv:2603.03331*, 2026.
- [8] H.-C. Lee, Y. Park, S. B. Yoon, S. M. Yang, D. Park, and C.-W. Jung, "Vitaldb, a high-fidelity multi-parameter vital signs database in surgical patients," *Scientific Data*, vol. 9, no. 1, p. 279, 2022.
- [9] A. Y. Hannun, P. Rajpurkar, M. Haghighpanahi, G. H. Tison, C. Bourn, M. P. Turakhia, and A. Y. Ng, "Cardiologist-level arrhythmia detection and classification in ambulatory electrocardiograms using a deep neural network," *Nature medicine*, vol. 25, no. 1, pp. 65–69, 2019.
- [10] A. Pillai, D. Spathis, F. Kawsar, and M. Malekzadeh, "Papagei: Open foundation models for optical physiological signals," in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 48 230–48 261.
- [11] M. Saha, M. A. Xu, W. Mao, S. Neupane, J. M. Rehg, and S. Kumar, "Pulse-ppg: An open-source field-trained ppg foundation model for wearable applications across lab and field settings," *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, vol. 9, no. 3, pp. 1–35, 2025.
- [12] M. Kachuee, M. Kiani, H. Mohammadzade, and M. Shabany, "Cuff-less blood pressure estimation," *UCI Machine Learning Repository*, vol. 10, p. C5B602, 2015.
- [13] A. Garde, P. Dehkordi, W. Karlen, D. Wensley, J. M. Ansermino, and G. A. Dumont, "Development of a screening tool for sleep disordered breathing in children using the phone oximeter™," *PloS one*, vol. 9, no. 11, p. e112959, 2014.
- [14] P. Schmidt, A. Reiss, R. Duerichen, C. Marberger, and K. Van Laerhoven, "Introducing wesad, a multimodal dataset for wearable stress and affect detection," in *Proceedings of the 20th ACM international conference on multimodal interaction*, 2018, pp. 400–408.
- [15] H. M. Pham, J. Tang, A. Saeed, and D. Ma, "Q-heart: Ecg question answering via knowledge-informed multimodal llms," *arXiv preprint arXiv:2505.06296*, 2025.
- [16] Y. Zhang, K. Ayush, S. Qiao, A. A. Heydari, G. Narayanswamy, M. Xu, A. Metwally, J. Xu, J. Garrison, X. Xu *et al.*, "Sensorlm: Learning the language of wearable sensors," *Advances in Neural Information Processing Systems*, vol. 38, pp. 46 813–46 849, 2026.